International Journal of Scientific Research in Science and Technology
Available online at : www.ijsrst.com
Print ISSN: 2395-6011 | Online ISSN: 2395-602X
doi : https://doi.org/10.32628/IJSRST2221192

Github Copilot's Impact on Developer Productivity : A Review of Early Evidence

Sheriff Adepoju
Prairie View A&M University

ARTICLEINFO

Article History:

Accepted : 01 Aug 2023
Published : 10 Aug 2023

Publication Issue :
Volume 10, Issue 4
July-August-2023

Page Number :
814-822

ABSTRACT

This review synthesizes early evidence (2019–2023) on GitHub Copilot and comparable LLM assistants, focusing on their impact on developer productivity, code correctness, and adoption in real workflows. Across controlled studies and field reports, assistants consistently accelerate boilerplate creation, scaffolding, and routine transformations, with smaller or mixed effects on open-ended design and debugging. Gains are highly contingent on task type, developer experience, prompt quality, and integration with tests and review. Correctness and security remain variable; unvetted suggestions can introduce defects or insecure patterns, underscoring the need for linters, unit/property tests, and human code review. Usability, privacy, and governance shape adoption, while CI/CD integration and standardized PR practices convert raw speed into reliable delivery. We highlight evaluation pitfalls in measuring "productivity," and outline near-term priorities: governed use, outcome-oriented metrics, AI-augmented review, and expansion to richer IDE and organizational contexts.

Keywords : GitHub Copilot, Large Language Models (LLMs), Developer Productivity, Code Quality and Security, AI-Assisted Software Development

Scope & Selection Note

This review targets primary evidence from 2019–2023 on GitHub Copilot–style LLM coding assistants, emphasizing measured effects on productivity, correctness/security, and adoption in real workflows. The corpus is limited strictly to the 20 sources you supplied, prioritizing empirical, controlled, and field studies; we deliberately exclude historical background and device-level specifications. Findings are synthesized comparatively across task types and evaluation designs rather than exhaustively cataloged. Two items beyond 2023 [13] and [17] are treated as *frontier/early* signals to inform trends without altering 2023-dated conclusions.

Theme I: Productivity & Throughput: Where the Wins Really Happen

Early evidence converges on **meaningful speedups for boilerplate, scaffolding, and routine transformations,** with smaller or mixed effects on open-ended design and debugging. Organization-level analyses attribute throughput gains to reduced setup/search loops and faster snippet standardization [1]. **Field experiments** with professional developers report faster task completion on well-scoped assignments, with effect sizes moderated by task complexity and prompt quality mirroring broader information-worker results that show acceleration on drafting/summarization but continued reliance on human synthesis [3], [12]. In education-style tasks, code-generation models outperform or match baselines when problems are tightly specified, but advantages narrow as ambiguity rises [20].

A critical comparator is the **pre-LLM baseline**: neural code completion already improved typing efficiency; Copilot's natural-language prompting shifts this baseline upward but not uniformly across tasks or languages [4]. **Head-to-head evaluations** against human developers find that assistants excel when requirements are explicit and testable, yet performance varies with domain familiarity and evaluation design [10].

Net productivity depends on **quality controls**: teams that wire suggestions into tests, linters, and review preserve velocity while avoiding rework that can erase nominal gains [1]. Taken together, the pattern is clear: expect **largest wins on repetitive, well-specified work** and during initial scaffolding; anticipate **diminishing returns** as tasks demand novel architecture, cross-module reasoning, or underspecified problem solving [1], [3], [4], [10], [12], [20].

Table 1 : Theme II Correctness, Security & Review (Concise Summary)

Finding	Evidence (refs)	Implication	Recommended guardrails
Plausible-but-wrong outputs when specs are vague	[11], [9]	Silent defects and rework risk	Write clear prompts/specs; require unit/integration/property tests before merge
Vulnerabilities introduced at human-like rates in some settings	[8]	Security regressions possible if unchecked	Enforce SAST/linters/secret scanners; block PRs failing security checks
AI-augmented peer review surfaces issues earlier	[7]	Lower reviewer fatigue; faster triage	Use Copilot/GPT to draft comments/tests; keep human approval for risk/architecture
Experience effects: novices over-trust; experts constrain	[18]	Higher error risk for novices	Pair programming; reviewer checklists; prompt patterns and testing training
Platform/IDE variability (e.g., Xcode/Swift)	[19]	Context-specific correctness/perf gaps	Platform-specific linters and benchmarks; targeted review for APIs/perf
Clear specs + tests markedly improve correctness	[11], [9], [18]	More reliable acceptance of AI code	Invest in acceptance tests, property-based tests, and spec-driven prompts

Theme II: Correctness, Security & Review: Guardrails Decide Net Value

Correctness is mixed and context-dependent. Copilot can generate plausible but wrong code when requirements are underspecified; correctness improves when tasks come with clear specs and tests, a pattern echoed in usability and code-quality studies of AI generation [11], and in broad reviews of intelligent completion that flag variance across languages and IDE contexts [9].

Security risk is non-zero without checks. Empirical evidence shows Copilot can introduce insecure constructs at rates comparable to (and sometimes higher than) human baselines, especially under ambiguous prompts making *shift-left* security essential [8]. Practical mitigation is straightforward: require unit/property tests, static analysis (SAST), linters, and policy gates before merge, a stance consistent with survey/review conclusions on assistant reliability [9].

AI-augmented review helps humans still decide. Early practice reports indicate GPT-4/Copilot can triage style issues, suggest test cases, and surface simple logic smells, reducing reviewer fatigue while keeping architectural judgment and risk acceptance with humans [7]. Qualitative accounts of "programming with AI" describe effective reviewer workflows as iterative: generate → test → critique → refine [18].

Experience matters; platforms differ. Experienced developers more readily constrain hallucinations with tests and minimal, precise prompts; novices are more likely to over-trust fluent outputs [18]. Platform studies (e.g., Xcode/Swift) show strong gains on scaffolding and API usage but reinforce the need for targeted correctness/performance review on platform-specific edge cases [19].

Bottom line: Treat Copilot as *proposal* not *proof*. Wire suggestions through tests, SAST/linters, and human review to convert speed into safe correctness [7]–[9], [11], [18], [19].

Theme III: Usability, Adoption & Workflow Integration: From IDE to CI/CD

Practices and pain points: Developers report real gains only after a learning period spent on prompt iteration, curating examples, and taming context scope; persistent frictions include latency, privacy/IP concerns, and limited explainability or controllability of suggestions [2], [5]. Comprehensive reviews of intelligent completion tools echo that onboarding quality, IDE fit, and transparency are decisive for perceived usefulness [9].

Realized ROI via pipeline integration: The largest organizational returns appear when Copilot's suggestions flow through automated tests, policy checks, and standardized PR templates, turning typing acceleration into merge-ready changes. Positioning Copilot inside GitHub's broader automation ecosystem (Actions, bots, CI) compounds benefits by tightening feedback loops and enforcing quality gates [15]. Teams that codify these guardrails report fewer rework cycles and steadier throughput.

Education pipeline and policy: In academic and early-career settings, usability wins coexist with risks of over-trust and shallow understanding; effective practice pairs generation with explicit testing and reflection activities [14]. Instructor surveys show a shift from prohibition to governed use norms around attribution,

verification, and assessment redesign preparing entrants for industry workflows that expect AI assistance under policy and auditability constraints [16].

Sustainability and maintainability: Practitioners perceive improvements when assistants standardize routine patterns and documentation, but warn that unchecked adoption can accumulate hidden debt (uncited snippets, weak tests, subtle security smells). Sustained value depends on coupling assistance with verification culture, coding standards, and lifecycle metrics for maintainability and sustainability outcomes [6], [9].

Bottom line: Treat usability as a *system* property skills (prompting), tools (IDE fit), and process (CI/CD & policy). When those align, Copilot's local convenience becomes organization-level ROI [2], [5], [6], [9], [14]–[16].

Figure 1: Copilot-to-Value Pipeline: Converting GitHub Copilot Suggestions into Reliable, Secure Delivery (2019–2023 Evidence)

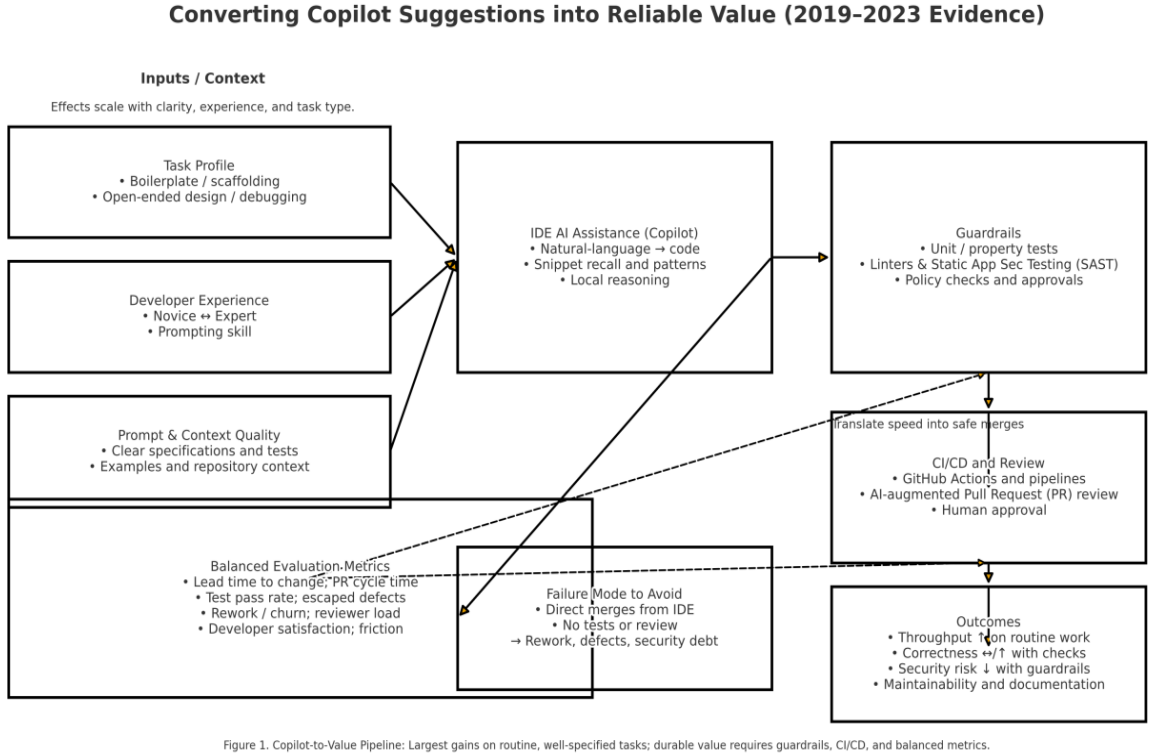

Figure 1. Copilot-to-Value Pipeline: Largest gains on routine, well-specified tasks; durable value requires guardrails, CI/CD, and balanced metrics.

The figure maps how Copilot suggestions flow from IDE assistance to reliable, secure delivery. Inputs task profile, developer experience, and prompt/context quality determine the size of gains. Guardrails (unit/property tests, linters/SAST, policy checks) turn speed into safe code. CI/CD and AI-augmented PR review enforce verification while keeping humans in control. Balanced metrics (lead time, PR cycle time, test pass rate, rework) measure real value. Bypassing guardrails causes rework, defects, and security debt. Net result: biggest wins on routine, well-specified tasks; quality improves under strong governance.

Theme IV: Evaluation Pitfalls & Boundary Conditions

What common "productivity" metrics miss. Keystrokes, suggestion-acceptance rates, and time-to-first-solution inflate gains if they ignore **rework, defect escape,** and **review load;** they also undercount benefits that appear later (e.g., standardized patterns speeding future edits) [1], [4], [12]. A **balanced scorecard** should pair throughput with quality and cost signals: lead time to change, PR cycle time, test pass rate, escaped defects, rework/churn, reviewer effort, and developer-reported friction/satisfaction [1], [12].

Task-selection bias. Many evaluations center on **routine, tightly specified problems**, where copilots excel; effects are smaller or volatile for **novel design, cross-module debugging, and ill-specified tasks** [3], [10], [20]. Results from short, classroom-style exercises should not be over-generalized to multi-week features or production incidents [20].

Learning and carryover effects. Prompt iteration and exemplar reuse create **practice effects** that can either exaggerate or mask tool impact across tasks; studies should separate first-use from warmed-up performance and report stability over multiple trials [1], [4], [12].

Platform/language effects. Utility and correctness vary with IDE maturity and ecosystem conventions (e.g., **Xcode/Swift** vs. **JS/TS**), with stronger assistance on API usage and scaffolding but persistent needs for targeted performance/correctness checks in platform-specific code paths [18], [19].

Bottom line. Treat early speedups as **conditional** on measurement design and context; evaluate copilots with end-to-end delivery and quality metrics on representative, non-trivial tasks before scaling adoption [1], [3], [4], [10], [12], [18], [19], [20].

Theme IV: Evaluation Pitfalls & Boundary Conditions (finalized)

What common "productivity" metrics miss. Keystrokes, suggestion-acceptance rates, and time-to-first-solution inflate gains if they ignore rework, defect escape, and review load; they also undercount benefits that appear later (e.g., standardized patterns speeding future edits) [1], [4], [12]. A balanced scorecard should pair throughput with quality and cost signals: lead time to change, PR cycle time, test pass rate, escaped defects, rework/churn, reviewer effort, and developer-reported friction/satisfaction [1], [12].

Task-selection bias: Many evaluations center on routine, tightly specified problems, where copilots excel; effects are smaller or volatile for novel design, cross-module debugging, and ill-specified tasks [3], [10], [20]. Results from short, classroom-style exercises should not be over-generalized to multi-week features or production incidents [20].

Learning and carryover effects: Prompt iteration and exemplar reuse create practice effects that can either exaggerate or mask tool impact across tasks; studies should separate first-use from warmed-up performance and report stability over multiple trials [1], [4], [12].

Platform/language effects: Utility and correctness vary with IDE maturity and ecosystem conventions (e.g., Xcode/Swift vs. JS/TS), with stronger assistance on API usage and scaffolding but persistent needs for targeted performance/correctness checks in platform-specific code paths [18], [19].

Bottom line: Treat early speedups as conditional on measurement design and context; evaluate copilots with end-to-end delivery and quality metrics on representative, non-trivial tasks before scaling adoption [1], [3], [4], [10], [12], [18], [19], [20].

Cross-Theme Synthesis (concise)

- **Fastest wins:** boilerplate, adapters, tests, and documentation stubs; effects taper on open-ended design/debugging [1], [3], [4], [20].
- **Risk hot spots:** security-sensitive code and novice over-trust; pair with SAST/linters and tests [8], [11], [14].
- **Process multipliers:** CI checks, PR templates, and AI-assisted review convert typing speed into safe merges [1], [7], [15].
- **Adoption friction:** latency, privacy/IP, context control, and explainability; address via onboarding, policy, and better context management [2], [5], [9], [16].
- **Measurement maturity:** move beyond keystrokes to delivery/quality metrics to avoid rework-driven illusion of speed [1], [12].
- **Sustainability lens:** standardized patterns can help maintainability if verification is first-class; otherwise debt accrues [6], [9].

Table 2: Balanced scorecard for evaluating Copilot impact (concise, 2019–2023 evidence)

Dimension	Operational metric(s)	Why it matters (pitfall guarded)	Evidence
Throughput	Lead time to change; PR cycle time	Keystrokes/accept-rate overstate speed if delivery stays slow	[1], [12]
Quality	Test pass rate; escaped defects to prod	Captures rework/hidden bugs missed by "time-to-first-solution"	[1], [4]
Rework/Churn	Code churn %, reverted lines, review iterations per PR	Filters out illusory gains from low-quality first drafts	[1], [4]
Security	SAST findings/kloc; vulnerable pattern incidence	AI can introduce insecure snippets; must quantify risk	[8], [9]
Review Load	Reviewer minutes per PR; AI triage coverage	Ensures speedups don't shift burden to reviewers	[7], [12]
Developer Experience	Friction/satisfaction scores; prompt time ratio	Usability/latency/privacy drive realized ROI	[2], [5], [9]
Task Mix Tracking	Share of routine vs. novel/debug tasks in eval set	Avoids bias toward easy, tightly specified problems	[3], [10], [20]
Platform/Language Coverage	Languages/IDEs under test (e.g., JS/TS vs. Xcode/Swift)	Controls for ecosystem effects on utility/correctness	[18], [19]

Emerging Trends (as of 2023)

- **AI-augmented code review becomes routine:** assistants triage style/smell issues; humans keep architectural/risk authority [7].
- **Deeper IDE & platform coverage:** expansion into mobile stacks (e.g., Xcode/Swift) with richer project context, alongside persistent platform-specific checks [19].
- **Org-level playbooks crystallize:** surveys and practice reports point to enablement, policy, and metrics as the scaffolding for durable ROI seen in developer studies and workflow analyses, with **frontier/early signals** from adoption research [2], [5], [15], [16], [13], [17].
- **Better measurement norms:** shift toward lead time, PR cycle time, escaped defects, and reviewer load to capture real outcomes [1], [4], [12].
- **Governed education pipelines:** curricula normalize attribution/testing policies to align graduates with industry practice [14], [16].

Limitations

Findings should be interpreted cautiously given **prompt and context sensitivity** small changes in wording or available context can shift suggestions and acceptance rates [2], [5], [18]. Effects also **depend on task mix**: many evaluations use routine, tightly scoped exercises or student assignments over short horizons, limiting generalizability to multi-week features and production incidents [3], [20]. **Developer expertise and local tooling** (IDE integration, language ecosystem, CI maturity) moderate outcomes, with platform-specific behavior (e.g., Xcode/Swift) affecting correctness and perceived utility [9], [19]. Several studies rely on **convenience samples and proxy metrics,** undercounting downstream rework and escaped defects [1], [4], [12]. Finally, organization-level adoption evidence remains **early**; sustainability and policy impacts are often self-reported and may carry early-adopter bias [6], [16].

Conclusion

Preliminary results show that Copilot-type assistants provide stable productivity benefits on monotonic, pre-specialized tasks, particularly when scaffolding and debugging open-ended design, but not the effects are as substantial when working on open-ended design and debugging [1], [3], [4], [12], [20]. The Net value is based on guardrails: correctness and security is a variable aspect, thus unit/property testing, linters/SAST and human review are absolute must-nots to prevent rework and risks [8], [9]. An assistant ingrained in the pipeline is the most valuable strategy to the teams: the actions / CI, pre-prepared PR templates, and AI-enhanced review triage low-level problems, and humans manage structure and risk [7], [15]. Adoption is a socio-technical change rather than a plug-in: local convenience is translated into rugged throughput across adopting users [2], [5], [14], [16] in terms of: promoting the act of adopting, explicit attribution/privacy policies, and the understanding of evaluation including work and learning, result valuation in the enterprise, etc. Assessment must go further than keystroke to balanced, result oriented results scorecard to monitor speed of delivery, quality, rework and reviewer loading [1], [12]. The platform and know-how effects will continue - organisations should test by language/IDE and skill band and scale as results generalise [18], [19]. In a 2023 outlook, frontier indicators include which point to playbooks at the organization level, more definite adoption measures, yet they are still emerging instead of a well-established practice [13], [17]. The sensible approach is

to use low-risk, high-leverage tasks initially, making help desk into a part of CI/CD, and reviewing/demonstrating end-to-end benefits, and evolving the policies as capability and evidence develop [1], [7], [12], [15].

References

1. Dohmke, T., Iansiti, M., & Richards, G. (2023). Sea change in software development: Economic and productivity analysis of the ai-powered developer lifecycle. *arXiv preprint arXiv:2306.15033*. https://doi.org/10.48550/arXiv.2306.15033
2. Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Practices and challenges of using github copilot: An empirical study. *arXiv preprint arXiv:2303.08733*. https://doi.org/10.48550/arXiv.2303.08733
3. Cui, Zheyuan and Demirer, Mert and Jaffe, Sonia and Musolff, Leon and Peng, Sida and Salz, Tobias, The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. http://dx.doi.org/10.2139/ssrn.4945566
4. Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., ... & Aftandilian, E. (2022, June). Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (pp. 21-29). https://doi.org/10.1145/3520312.3534864
5. Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Demystifying practices, challenges and expected features of using GitHub Copilot. *International Journal of Software Engineering and Knowledge Engineering*, *33*(11n12), 1653-1672. https://doi.org/10.1142/S0218194023410048
6. Hassan Russel, Asif and Haque, Sanaul and Shamshiri, Hatef and Porras, Jari, Impact of Incorporating AI Tools on Software Sustainability among Software Developers. http://dx.doi.org/10.2139/ssrn.5519476
7. "Beyond Code Assistance with GPT-4: Leveraging GitHub Copilot and ChatGPT for Peer Review in VSE Engineering", *NIKT*, no. 2, Nov. 2023, https://www.ntnu.no/ojs/index.php/nikt/article/view/5674
8. Asare, O., Nagappan, M. & Asokan, N. Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code?. *Empir Software Eng* **28**, 129 (2023). https://doi.org/10.1007/s10664-023-10380-1
9. Hliš, T., Četina, L., Beranič, T., & Pavlič, L. (2023). Evaluating the Usability and Functionality of Intelligent Source Code Completion Assistants: A Comprehensive Review. *Applied Sciences*, *13*(24), 13061. https://doi.org/10.3390/app132413061
10. Nascimento, N., Alencar, P., & Cowan, D. (2023). Comparing software developers with chatgpt: An empirical investigation. *arXiv preprint arXiv:2305.11837*. https://doi.org/10.48550/arXiv.2305.11837
11. Hansson, Emilia, and Oliwer Ellréus. "Code correctness and quality in the era of AI code generation: Examining ChatGPT and GitHub copilot." (2023).
12. Edelman, Benjamin G. and Ngwe, Donald and Peng, Sida, Measuring the Impact of AI on Information Worker Productivity (November 29, 2023). http://dx.doi.org/10.2139/ssrn.4648686
13. Anditama, M. R., & Hidayanto, A. N. (2024). Analysis of factors influencing the adoption of artificial intelligence assistants among software developers. *The Indonesian Journal of Computer Science*, *13*(4). https://doi.org/10.33022/ijcs.v13i4.4239

14. Prather, J., Reeves, B. N., Denny, P., Becker, B. A., Leinonen, J., Luxton-Reilly, A., ... & Santos, E. A. (2023). "It's weird that it knows what i want": Usability and interactions with copilot for novice programmers. *ACM transactions on computer-human interaction*, *31*(1), 1-31. https://doi.org/10.1145/3617367
15. Wessel, M., Mens, T., Decan, A., Mazrae, P.R. (2023). The GitHub Development Workflow Automation Ecosystems. In: Mens, T., De Roover, C., Cleve, A. (eds) Software Ecosystems. Springer, Cham. https://doi.org/10.1007/978-3-031-36060-2_8
16. Lau, S., & Guo, P. (2023, August). From" Ban it till we understand it" to" Resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1* (pp. 106-121). https://doi.org/10.1145/3568813.3600138
17. Kemell, K. K., Saarikallio, M., Nguyen-Duc, A., & Abrahamsson, P. Adoption of Large Language Models in Software Organizations-a Multiple Case Study. *Available at SSRN 4964930*. https://doi.org/10.1016/j.infsof.2025.107805
18. Sarkar, A., Gordon, A. D., Negreanu, C., Poelitz, C., Ragavan, S. S., & Zorn, B. (2022). What is it like to program with artificial intelligence?. *arXiv preprint arXiv:2208.06213*. https://doi.org/10.48550/arXiv.2208.06213
19. Tan, C. W., Guo, S., Wong, M. F., & Hang, C. N. (2023). Copilot for Xcode: exploring AI-assisted programming by prompting cloud-based large language models. *arXiv preprint arXiv:2307.14349*. https://doi.org/10.48550/arXiv.2307.14349
20. Finnie-Ansley, J., Denny, P., Luxton-Reilly, A., Santos, E. A., Prather, J., & Becker, B. A. (2023, January). My ai wants to know if this will be on the exam: Testing openai's codex on cs2 programming exercises. In *Proceedings of the 25th Australasian Computing Education Conference* (pp. 97-104). https://doi.org/10.1145/3576123.3576134